Name: Vaishnavi Bandaru

SRN: PES2UG24CS570

Section: J

Subject: Software Engineering

LAB 7: Fuzz Testing with Hypothesis

1. Objective

To perform fuzz testing on a Python application using the Hypothesis library to identify input validation bugs, handle unexpected inputs, and improve the robustness of the application.

2. Tools and Technologies Used

- Python
- Hypothesis – Property-based fuzz testing
- Pytest – Test execution framework
- VS Code – Development environment

3. Implementation and Testing

Step 1: Execution of Original Application

The original processor.py application was executed using valid inputs to verify the functionality of its three functions:

- parse_transit_corridor()
- lookup_concession_discount()
- compute_passenger_density()

Screenshot 1: Original Application Execution

```
(venv) PS D:\College\S5\SE\Labs\Lab 6\L6_L7_10_bus_transit_fare\Lab 7> python processor.py
==== Running parse_transit_corridor ====
Enter corridor (e.g. DT-UP-3): DT-UP-3
Route DT->UP: 3 Zones, Fare $4.75

==== Running lookup_concession_discount ====
Enter concession category (e.g. STUDENT, SENIOR): STUDENT
Multiplier: 0.5

==== Running compute_passenger_density ====
Enter passengers/km (e.g. 50/10.0): 50/10.0
Density: 5.0
(venv) PS D:\College\S5\SE\Labs\Lab 6\L6_L7_10_bus_transit_fare\Lab 7>
```

Step 2: Implementing Hypothesis Tests

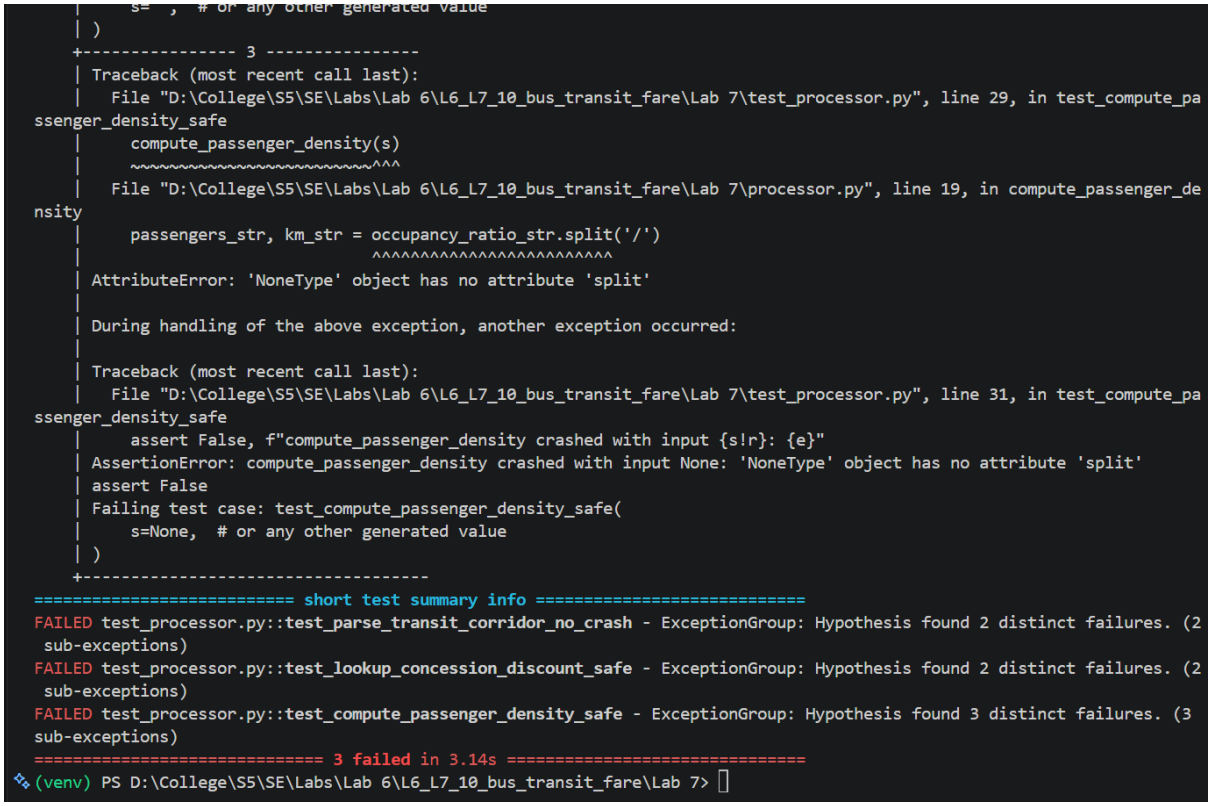
The test_processor.py file was completed using Hypothesis. The @given decorator and st.text() | st.none() strategy were used to automatically generate different inputs, including unexpected and invalid values.

Exception handling was implemented to identify inputs that caused the functions to crash.

Step 3: Identifying Bugs Through Fuzz Testing

The tests were executed using pytest. Hypothesis generated multiple inputs and identified edge cases that caused exceptions in the original implementation.

Screenshot 2: Failing Hypothesis Tests



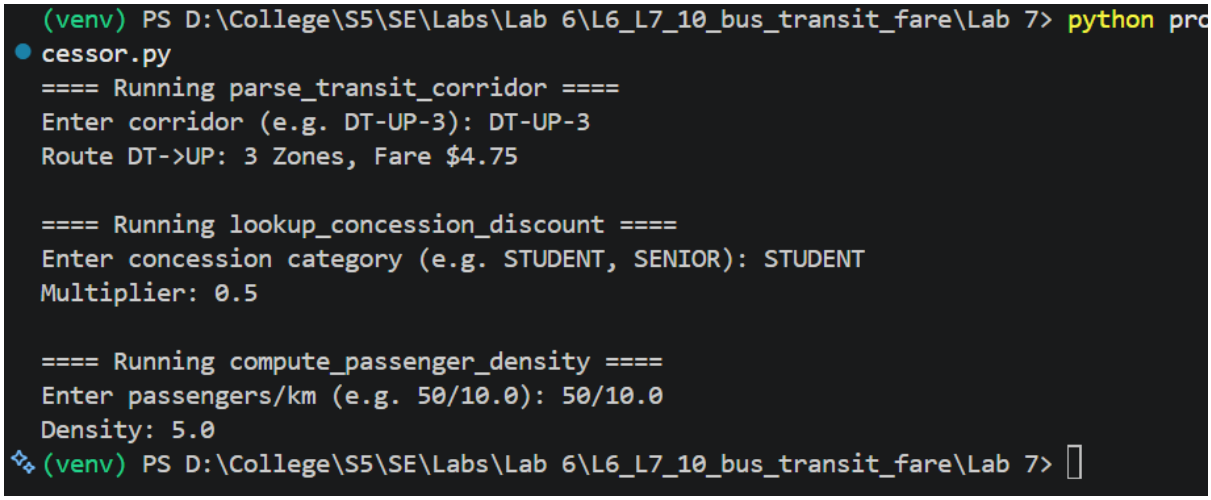
Step 4: Fixing the Application

The identified input validation issues were addressed in processor.py.

The functions were modified to validate input types, handle malformed strings, check numeric values, prevent invalid dictionary lookups, and avoid division by zero.

Invalid inputs are handled safely without causing unexpected crashes.

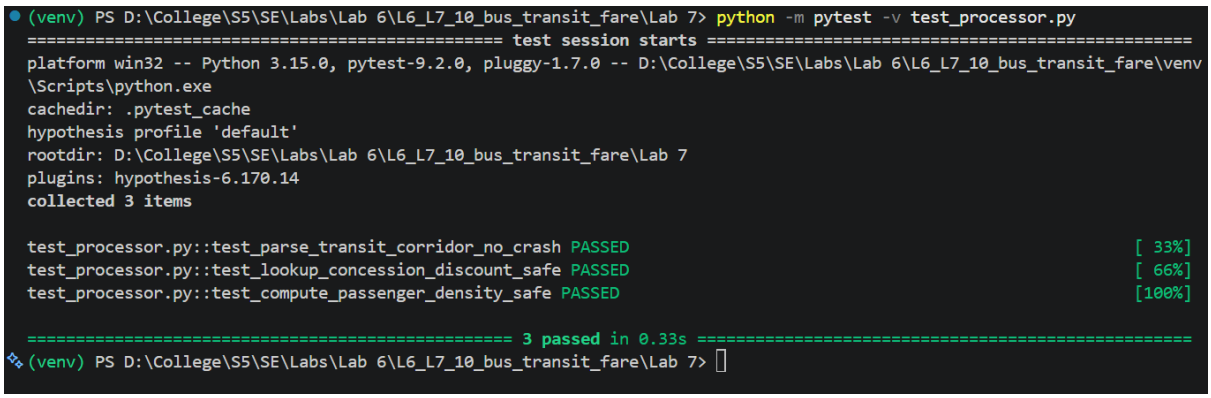
Screenshot 3: Execution of Fixed Application



Step 5: Verification Using Hypothesis

The Hypothesis tests were executed again against the modified application using pytest. The tests passed successfully, demonstrating improved handling of unexpected inputs.

Screenshot 4: Successful Test Execution



4. Conclusion

Fuzz testing using Hypothesis helped identify input validation bugs that could be missed through normal testing. By automatically generating unexpected inputs and analysing failures, the application was made more robust through defensive programming. The successful execution of the tests after fixing the bugs verified the improved reliability of the application.